

Exemple pratique DDD avec MongoDB (Node.js)

Ce document montre un exemple concret **Domain-Driven Design (DDD)** appliqué à une petite application de gestion de tâches (Task Management). L'objectif : illustrer les concepts de base (Bounded Context, Entités, Value Objects, Agrégats, Repositories, Domain Services, Domain Events) et fournir un squelette de projet Node.js qui utilise MongoDB comme infrastructure de persistance.

Contexte métier

Application : **TaskFlow** — gérer des tâches pour une équipe. Fonctions principales : créer des tâches, attribuer une tâche, changer le statut, ajouter des commentaires, prioriser.

Bounded Contexts : - **User Context** : création / gestion des utilisateurs - **Task Context** : tout le domaine des tâches (agrégat Task)

Ubiquitous Language : Task, User, Assignee, Status, Priority, Comment, Aggregate Root, Repository.

Arborescence proposée (squelette)

```
taskflow/
├─ src/
│   ├─ domain/
│   │   ├─ task/
│   │   │   ├─ Task.js           # Aggregate root + règles métier
│   │   │   ├─ TaskId.js        # Value Object
│   │   │   ├─ TaskStatus.js    # Value Object / Enum
│   │   │   └─ events/          # Domain events (TaskCreated,
│   │   │       TaskAssigned...)
│   │   └─ user/
│   │       ├─ User.js
│   │       └─ UserId.js
│   └─ application/
│       ├─ services/
│       │   └─ TaskCommandService.js # Use-cases / orchestration
│       └─ dtos/
│   └─ infrastructure/
│       ├─ db/
│       │   └─ mongoClient.js    # connexion MongoDB
│       ├─ repositories/
│       │   └─ TaskRepositoryMongo.js
│       └─ events/
│   └─ interfaces/
│       └─ http/
│           └─ controllers/
│               └─ tasksController.js
```

```
| |      └ routes.js
| └ index.js
└ package.json
```

1) Domain layer — exemples

TaskId (Value Object)

```
// src/domain/task/TaskId.js
const { v4: uuidv4 } = require('uuid')

class TaskId {
  constructor(id) {
    this.id = id || uuidv4()
  }

  toString() { return this.id }
}

module.exports = TaskId
```

TaskStatus (Value Object / Enum)

```
// src/domain/task/TaskStatus.js
const Status = Object.freeze({ TODO: 'todo', DOING: 'doing', DONE: 'done' })
module.exports = Status
```

Task (Aggregate Root)

```
// src/domain/task/Task.js
const TaskId = require('./TaskId')
const Status = require('./TaskStatus')

class Task {
  constructor({ id, title, description, assigneeId = null, priority =
'normal', status = Status.TODO, comments = [], createdAt = new Date() }) {
    this.id = id instanceof TaskId ? id : new TaskId(id)
    this.title = title
    this.description = description
    this.assigneeId = assigneeId
    this.priority = priority
    this.status = status
    this.comments = comments
    this.createdAt = createdAt
    this._events = [] // domain events produced
  }
}
```

```

// règles métier
assignTo(userId) {
  if (!userId) throw new Error('AssigneeRequired')
  this.assigneeId = userId
  this._events.push({ type: 'TaskAssigned', payload: { taskId:
this.id.toString(), assigneeId: userId } })
}

changeStatus(newStatus) {
  if (!Object.values(Status).includes(newStatus)) throw new
Error('InvalidStatus')
  this.status = newStatus
  this._events.push({ type: 'TaskStatusChanged', payload: { taskId:
this.id.toString(), status: newStatus } })
}

addComment(authorId, text) {
  if (!text) throw new Error('CommentEmpty')
  const comment = { id: `${Date.now()}`, authorId, text, createdAt: new
Date() }
  this.comments.push(comment)
  this._events.push({ type: 'TaskCommentAdded', payload: { taskId:
this.id.toString(), comment } })
}

pullEvents() { const e = this._events; this._events = []; return e }

static fromPersistence(raw) {
  return new Task({ id: raw._id, title: raw.title, description:
raw.description, assigneeId: raw.assigneeId, priority: raw.priority, status:
raw.status, comments: raw.comments || [], createdAt: raw.createdAt })
}
}

module.exports = Task

```

Notes : l'agrégat Task encapsule les invariants liés à la tâche (ex : statut valide, assignee requis pour certaines transitions, etc.).

2) Repository interface (contrat)

Dans DDD, le repository est un contrat côté domaine. Voici un exemple d'interface (pseudo) que l'on respecte dans l'infra :

```

// src/domain/task/ITaskRepository.js (pseudo-interface)
class ITaskRepository {
  async save(task) { throw new Error('NotImplemented') }
}

```

```

    async findById(taskId) { throw new Error('NotImplemented') }
    async findByAssignee(assigneeId) { throw new Error('NotImplemented') }
  }
  module.exports = ITaskRepository

```

3) Infrastructure — implémentation MongoDB

MongoDB Client (connexions)

```

// src/infrastructure/db/mongoClient.js
const { MongoClient } = require('mongodb')
const uri = process.env.MONGO_URI || 'mongodb://localhost:27017'
const client = new MongoClient(uri)

async function connect() {
  if (!client.isConnected()) await client.connect()
  return client.db('taskflow')
}

module.exports = { connect }

```

Remarque : pour production préférez `MongoClient.connect()` une fois au démarrage et réutiliser l'instance. Utilisez des pools et MONGO_URI sécurisé.

TaskRepositoryMongo

```

// src/infrastructure/repositories/TaskRepositoryMongo.js
const Task = require('../../domain/task/Task')
const TaskId = require('../../domain/task/TaskId')

class TaskRepositoryMongo {
  constructor(db) { this.db = db; this.col = db.collection('tasks') }

  async save(task) {
    const raw = { _id: task.id.toString(), title: task.title, description:
task.description, assigneeId: task.assigneeId, priority: task.priority,
status: task.status, comments: task.comments, createdAt: task.createdAt }
    await this.col.updateOne({ _id: raw._id }, { $set: raw }, { upsert:
true })
  }

  async findById(taskId) {
    const raw = await this.col.findOne({ _id: taskId.toString() })
    if (!raw) return null
    return Task.fromPersistence(raw)
  }
}

```

```

    async findByAssignee(assigneeId) {
      const cursor = await this.col.find({ assigneeId }).toArray()
      return cursor.map(raw => Task.fromPersistence(raw))
    }
  }

  module.exports = TaskRepositoryMongo

```

4) Application layer — Use case / Command Service

Le service d'application orchestre l'usage du repository et applique des règles de domaine.

```

// src/application/services/TaskCommandService.js
class TaskCommandService {
  constructor({ taskRepository, eventBus }) {
    this.taskRepository = taskRepository
    this.eventBus = eventBus
  }

  async createTask({ title, description, priority }) {
    const Task = require('../../domain/task/Task')
    const task = new Task({ title, description, priority })
    await this.taskRepository.save(task)
    const events = task.pullEvents()
    await this.eventBus.publish(events)
    return task
  }

  async assignTask({ taskId, userId }) {
    const task = await this.taskRepository.findById(taskId)
    if (!task) throw new Error('TaskNotFound')
    task.assignTo(userId)
    await this.taskRepository.save(task)
    await this.eventBus.publish(task.pullEvents())
    return task
  }

  async addComment({ taskId, authorId, text }) {
    const task = await this.taskRepository.findById(taskId)
    if (!task) throw new Error('TaskNotFound')
    task.addComment(authorId, text)
    await this.taskRepository.save(task)
    await this.eventBus.publish(task.pullEvents())
    return task
  }
}

```

```
module.exports = TaskCommandService
```

5) Event Bus simple (Infra)

```
// src/infrastructure/events/eventBusSimple.js
class EventBusSimple {
  constructor() { this.handlers = [] }
  subscribe(handler) { this.handlers.push(handler) }
  async publish(events) {
    for (const e of events) {
      for (const h of this.handlers) await h(e)
    }
  }
}
module.exports = EventBusSimple
```

Utilisé pour pousser des notifications, logs, projections, etc.

6) Interfaces — Controller HTTP (Express)

```
// src/interfaces/http/controllers/tasksController.js
const express = require('express')

function makeTasksController({ taskCommandService }) {
  const router = express.Router()

  router.post('/', async (req, res) => {
    try {
      const task = await taskCommandService.createTask(req.body)
      res.status(201).send({ id: task.id.toString(), title: task.title })
    } catch (err) { res.status(400).send({ error: err.message }) }
  })

  router.post('/:id/assign', async (req, res) => {
    try {
      const task = await taskCommandService.assignTask({ taskId:
req.params.id, userId: req.body.userId })
      res.send({ id: task.id.toString(), assignee: task.assigneeId })
    } catch (err) { res.status(400).send({ error: err.message }) }
  })

  router.post('/:id/comments', async (req, res) => {
    try {
      const task = await taskCommandService.addComment({ taskId:
```

```

req.params.id, authorId: req.body.authorId, text: req.body.text })
    res.status(201).send({ comments: task.comments })
  } catch (err) { res.status(400).send({ error: err.message }) }
})

return router
}

module.exports = makeTasksController

```

7) Démarrage de l'app (bootstrap)

```

// src/index.js
const express = require('express')
const { connect } = require('./infrastructure/db/mongoClient')
const TaskRepositoryMongo = require('./infrastructure/repositories/
TaskRepositoryMongo')
const EventBusSimple = require('./infrastructure/events/eventBusSimple')
const TaskCommandService = require('./application/services/
TaskCommandService')
const makeTasksController = require('./interfaces/http/controllers/
tasksController')

async function main() {
  const db = await connect()
  const taskRepo = new TaskRepositoryMongo(db)
  const eventBus = new EventBusSimple()

  // Exemple: subscribe to events
  eventBus.subscribe(async (evt) => console.log('Event reçu:', evt))

  const taskService = new TaskCommandService({ taskRepository: taskRepo,
eventBus })

  const app = express()
  app.use(express.json())
  app.use('/tasks', makeTasksController({ taskCommandService: taskService }))

  app.listen(3000, () => console.log('TaskFlow listening on :3000'))
}

main().catch(console.error)

```

8) Transactions et consistance

- Utiliser les transactions MongoDB (sessions) pour des cas multi-documents (ex : transfert de propriété, opérations financières). Dans cet exemple simple, chaque tâche est mono-document.
 - Si vous avez des invariants couvrant plusieurs collections, encapsulez-les via application service et utilisez `session.startTransaction()` dans l'infra.
-

9) Tests unitaires (exemples)

- Tester l'agrégat Task (règles métier) sans DB (pure unit test)
- Mock du repository pour tester le service d'application

Exemple simple (jest) :

```
const Task = require('../../src/domain/task/Task')

test('assign to sets assignee and emits event', () => {
  const task = new Task({ title: 't', description: 'd' })
  task.assignTo('user-1')
  expect(task.assigneeId).toBe('user-1')
  const events = task.pullEvents()
  expect(events.some(e => e.type === 'TaskAssigned')).toBe(true)
})
```

10) Projections / Read models

Pour les lectures optimisées, créez des projections (read models) alimentées par les domain events. Par exemple une collection `task_read` pré-calculée.

11) Bonnes pratiques DDD appliquées

- **Séparer** clairement Domain / Application / Infra / Interfaces
 - **Modéliser** les invariants dans les agrégats
 - **Repository** retourne des agrégats, cache la persistance
 - **EventBus** découple effets secondaires
 - **Keep it modular** : chaque Bounded Context dans un package/repo séparé si nécessaire
-

12) Extensions possibles

- CQRS : séparer writes (command handlers) des reads (projections + query handlers)
- Event sourcing : stocker uniquement les events et reconstruire l'agrégat
- Utiliser TypeScript pour types forts
- Ajouter validation (class-validator), logging, monitoring

Conclusion

Ce guide fournit un exemple concret de mise en œuvre des bases du DDD avec MongoDB. Tu peux : - Transformer l'exemple en projet réel - Ajouter validation, sécurité (JWT), webhooks, notifications - Étendre avec CQRS + Event Sourcing si besoin

Si tu veux, je peux maintenant : - Générer le code complet prêt à exécuter (package.json, scripts, dépendances) - Convertir en TypeScript - Ajouter tests unitaires et d'intégration (avec MongoDB Memory Server) - Créer un TP pas à pas expliquant chaque fichier

Quelle suite souhaites-tu ?